Ciências ULisboa | Informática

di.ciencias.ulisboa.pt

/difcul

Segurança e Confiabilidade

2025/2026

Trabalho 1 – 1ª fase

1 Objetivos

A parte prática da disciplina de Segurança e Confiabilidade pretende familiarizar os alunos com alguns dos problemas envolvidos na programação de aplicações distribuídas seguras, nomeadamente a gestão de chaves criptográficas, a geração de sínteses seguras, atestação remota, cifras e assinaturas digitais, e a utilização de canais seguros. O primeiro trabalho a desenvolver na disciplina será realizado utilizando a linguagem de programação Java e a API de segurança do Java, e é composto por duas fases. A primeira fase do projeto está enunciada neste documento, e tem como objetivo fundamental a construção de uma aplicação distribuída, focando-se essencialmente nas funcionalidades da aplicação. Na fase 2 do projeto, iremos dedicar os nossos esforços para garantir que todos os processos – transmissão de mensagens, salvaguarda de dados, ou execução de software – são seguros e confiáveis. O enunciado da segunda fase será divulgado oportunamente.

O trabalho envolve a implementação do *Sperta*, um sistema de casas inteligentes (smart homes) que permite gerir os seus dispositivos de domótica (smart things). O sistema assenta numa arquitetura cliente-servidor, como ilustra a Fig.1. A aplicação cliente do *Sperta*, denominada ***SpertaClient***, permite a um utilizador se conectar ao servidor e gerir as suas casas inteligentes, ou seja, enviar comandos para o servidor e receber dados deste. Para o cliente interagir com o servidor, primeiramente o utilizador terá de se autenticar neste, e posteriormente indicar qual a `Casa` que pretende gerir.

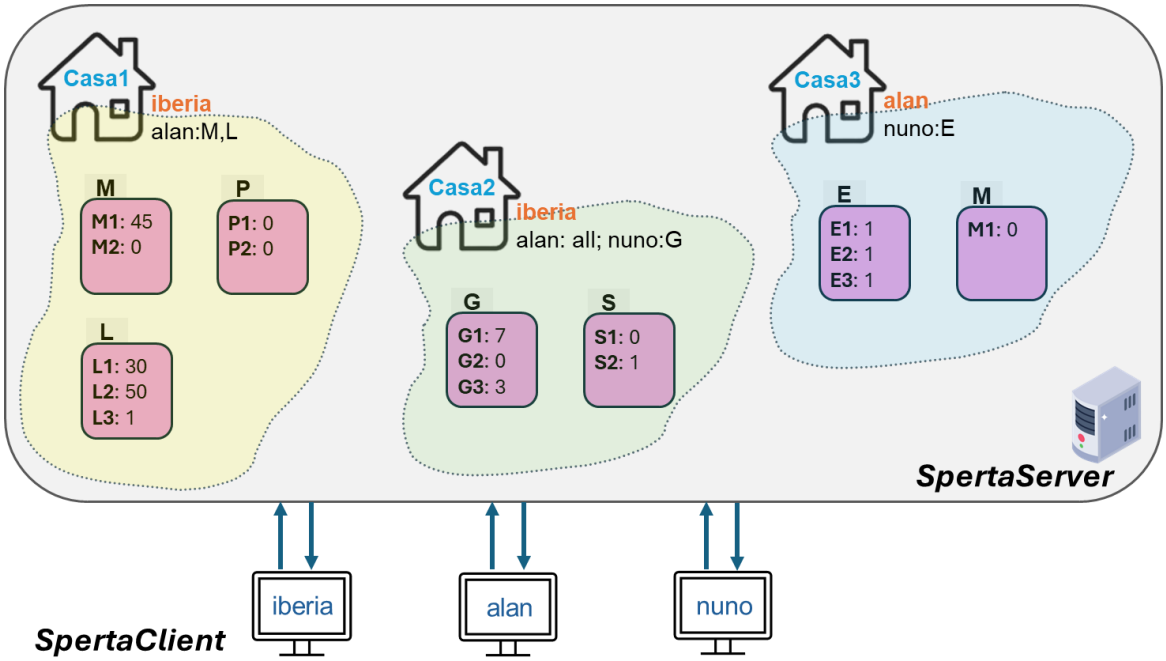

Fig.1. Diagrama da arquitetura do sistema ***Sperta***, com um Servidor e múltiplos clientes.

No servidor, uma `Casa` é vista como uma estrutura dividida em seções (ver Fig. 2), nas quais são inseridos os dispositivos inteligentes. As seções são vistas como representações de categoria de dispositivos (ex., Electrodomésticos, Garden, Luzes, Multimédia,...). Por exemplo, na Fig. 1, o utilizador `iberia` gere (i.e., é dono) `Casa1` e `Casa2`. A `Casa1` possui dispositivos inteligentes de **M**ultimédia, **P**ortas e **L**uzes; **M**ultimédia contém 2 dispositivos: `M1` e `M2` (ex., `M1` é uma TV e `M2` um SoundBar); `M1` está ligado por 45 min (`M1:45`) e `M2` está desligado (`M2:0`). Um utilizador, dono de uma casa, pode dar permissões a um outro utilizador para gerir seções da sua casa ou mesmo a casa inteira. Por exemplo, na `Casa1`, o utilizador `alan` tem permissões para gerir os dispositivos de **M**ultimédia e **L**uzes (`alan: M,L`). Na `Casa2`, o utilizador `alan` tem permissões para gerir toda a casa (`alan:all`), enquanto o utilizador `nuno` os dispositivos de **G**arden (`nuno:G`).

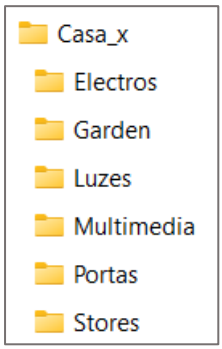

Fig.2. Estrutura de uma Casa no ***Sperta***.

A aplicação ***SpertaClient*** tem duas grandes funcionalidades: 1) Enviar comandos para o servidor para gerir os dispositivos (ligar, ligar com temporização e desligar) e dar permissões a utilizadores; 2) Aceder à informação sobre o último comando enviado para os dispositivos de uma dada casa, ou receber um ficheiro de log com o histórico dos comandos enviados a um dispositivo de uma dada casa. Para fazer uso de todas estas funcionalidades, a aplicação ***SpertaClient*** deverá implementar uma linha de comandos simples, para interação com o utilizador.

A aplicação servidor do *Sperta*, denominada ***SpertaServer***, permite a ligação com vários clientes em simultâneo, mantém informação (atual e histórica, na forma de log) sobre os dispositivos, casas e utilizadores registados, autentica e identifica utilizadores, e vai colecionando e partilhando informação já recebida dos vários clientes, de forma organizada e persistente. Cada dispositivo tem um ficheiro de log, no formato csv, o qual contém o seu histórico de funcionamento, ou seja, regista todos os comandos de operação (ligado, ligado com temporização ou desligado) do dispositivo, incluindo a data e hora da sua ocorrência.

2 Arquitetura do Sistema

O trabalho consiste no desenvolvimento de duas aplicações:

- A aplicação servidor ***SpertaServer*** (espera ligações TCP)
- A aplicação cliente ***SpertaClient*** (acede ao servidor via TCP)

O servidor deverá poder executar numa máquina e permitir ligar um número elevado de clientes a serem executados simultaneamente em máquinas distintas. A execução das aplicações deve ser feita da seguinte forma:

1. Servidor:

   `SpertaServer [port]`

   - `[port]` identifica o porto (TCP) para aceitar ligações de clientes. Por omissão, o servidor deve usar o porto 22345 e aceitar ligações em qualquer interface.

2. Cliente:

   SpertaClient <serverAddress> <user-id> <password>

   Em que:

   - <serverAddress> identifica o servidor. O formato de serverAddress é o seguinte: <IP/hostname>[:Port]. O endereço IP/hostname do servidor é obrigatório e o porto é opcional. Por omissão, o cliente deve ligar-se ao porto 22345 do servidor.
   - <user-id> - string que identifica o utilizador no servidor ***SpertaServer***.
   - <password> – string correspondente à password do utilizador.

3 Funcionalidades

Cliente

A aplicação cliente deve ter o seguinte comportamento sequencial:

1. Começa por enviar o user-id e a password ao servidor, para tentar autenticar o utilizador.
2. Um dos três cenários pode ocorrer:
   a. O cliente não consegue autenticar no servidor – este receberá uma mensagem do tipo WRONG-PWD. O utilizador terá oportunidade de voltar a tentar outra password.
   b. O cliente não existe registado no servidor – este será registado no servidor, e recebe uma mensagem do tipo OK-NEW-USER.
   c. O cliente existe e a password está correta – o cliente recebe uma mensagem do tipo OK-USER.
3. Após uma autenticação bem-sucedida, a aplicação cliente deverá apresentar um menu com os comandos disponíveis ao utilizador, nomeadamente o seguinte texto:

- **CREATE <hm>** # Criar casa <hm> - utilizador é Owner
- **ADD <user1> <hm> <s>** # Adicionar utilizador <user1> à casa <hm>, seção <s>.
- **RD <hm> <s>** # **R**egistar um **D**ispositivo na casa <hm>, na seção <s>
- **EC <hm> <d> <int>** # **E**nviar valor <int> de estado/temporização, do dispositivo <d> da casa <hm>, para o servidor.
- **RT <hm>**# **R**eceber a informação sobre o último comando (estados/temporizações) enviado a cada dispositivo da casa <hm>, desde que o utilizador tenha permissões.
- **RH <hm> <d>**# **R**eceber o **H**istórico (ficheiro de log .csv) de comandos enviados ao dispositivo <d> da casa <hm>, desde que o utilizador tenha permissões.

4. A aplicação cliente deverá implementar uma linha de comandos que permita a utilização dos comandos apresentados no passo 3.
5. O utilizador introduz um dos comandos disponíveis, e um pedido respetivo é enviado ao servidor. O cliente recebe a resposta do servidor e imprime essa informação no terminal. No caso dos comandos **RT** e **RH** o cliente também guarda localmente o ficheiro recebido.

   Enquanto o utilizador não pressionar CTRL+C, o programa deverá voltar ao passo 5. Caso contrário, termina.

Comandos Disponíveis

O comando CREATE <hm> deverá criar uma nova casa no servidor.

Caso o servidor aceite o pedido, o cliente deve receber uma mensagem do tipo OK.

Caso a casa já exista, deverá receber uma mensagem do tipo NOK.

```
Exemplo
Comando: CREATE Casa1
Resposta: OK
Comando: CREATE Casa1
Resposta: NOK # esta casa já existe
```

O comando **ADD <user1> <hm> <s>** deve permitir adicionar o utilizador <user1> à casa <hm>, seção <s>. As Seções são identificadas pela 1ª letra do seu nome (conforme a Fig. 2). Para dar permissões totais sobre a casa, então s = all.

Caso o servidor aceite o pedido, o cliente deverá receber uma mensagem do tipo OK.

Se o Utilizador que faz o pedido não é o dono (criador) da casa , a casa não existe, ou o utilizador não existe, então o cliente deverá receber uma mensagem do tipo NOPERM, NOHM, ou NOUSER, respetivamente.

```
Exemplo:
Comando: ADD alan Casa1 M
Resposta: OK
Comando: ADD alan Casa2 all
Resposta: OK
Comando: ADD alan Casa2 G
Resposta: NOPERM # sem permissões, supondo que o comando é enviado por um
                   utilizador que não é o dono da Casa2
Comando: ADD alan Casa4 P
Resposta: NOHM # esta casa não existe
Comando: ADD john Casa1 M
Resposta: NOUSER # este user não existe
```

O comando **RD <hm> <s>** permite registar um dispositivo na casa <hm>, na seção <s>. Somente o dono da casa é quem pode adicionar dispositivos.

Caso o servidor aceite o pedido, o cliente deve receber uma mensagem do tipo OK.

Caso o utilizador não seja o dono da casa <hm>, ou a casa não exista, o cliente deverá receber uma mensagem do tipo NOPERM ou NOHM, respetivamente.

```
Exemplo:
Comando: RD Casa1 P
```

Resposta: OK

Comando: **RD Casa2 P**

Resposta: NOPERM # sem permissões, supondo que o comando é enviado por um utilizador que não é o dono da Casa2

Comando: **RD Casa4 P**

Resposta: NOHM # esta casa não existe

O comando **EC <hm> <d> <int>** deve enviar o valor <int> de estado/temporização, como um número inteiro, que permite gerir o dispositivo <d> da casa <hm>. Valores possíveis: 0 (desligar), 1(ligar),]1..600] (ligar por x minutos, até 600).

Caso o servidor aceite a informação, o cliente deve receber uma mensagem do tipo OK.

Caso o pedido seja inválido, o cliente poderá receber uma mensagem do tipo NOK, NOHM, NOD ou NOPERM do servidor, indicando, respectivamente, valor <int> inválido, casa não existente, dispositivo inválido/inexistente, ou utilizador sem permissões para operar sobre o dispositivo.

Exemplo:

Comando: **EC Casa1 M1 45**

Resposta: OK

Comando: **EC Casa2 B1 batatas**

Resposta: NOK # valor inválido

Comando: **EC Casa4 M2 0**

Resposta: NOHM # esta casa não existe

Comando: **EC Casa2 B3 1**

Resposta: NOD # este dispositivo não existe

Comando: **EC Casa1 L2 1**

Resposta: NOPERM # sem permissões, supondo que o comando é enviado por um utilizador que não pertence à Casa1

O comando **RT <hm>** permite à aplicação cliente obter um ficheiro de texto com os últimos comandos de alteração de estado/temporização gravados no servidor referentes a todos os dispositivos de uma casa <hm>, ou aqueles que o utilizador tem permissão. Assim, o cliente deverá receber uma mensagem OK, depois um número LONG com o tamanho do ficheiro, e em seguida o conteúdo do ficheiro.

Caso o pedido seja inválido, o cliente poderá receber uma mensagem NODATA, NOHM ou NOPERM por parte do servidor, indicando, respetivamente, que os dispositivos ainda não receberam comandos, casa não existente, ou utilizador sem permissões para operar sobre a casa.

Exemplo:

Comando: **RT Casa1**

Resposta: OK, 65 (long), seguido de *65 bytes de dados.*

Comando: **RT Casa2**

Resposta: NODATA # não há dados para enviar

Comando: **RT Casa4**

Resposta: NOHM # esta casa não existe

Comando: **RT Casa2**

Resposta: NOPERM # sem permissões

O comando **RH <hm> <d>** permite à aplicação cliente obter o histórico (ficheiro de log .csv) do dispositivo <d> da casa <hm>. Assim, o cliente deverá receber uma mensagem OK, depois um número LONG com o tamanho do ficheiro, e em seguida o conteúdo do ficheiro.

Caso o pedido seja inválido, o cliente poderá receber uma mensagem NODATA, NOHM, NOD ou NOPERM por parte do servidor, indicando, respetivamente, que os dispositivos ainda não receberam comandos, casa não existente, dispositivo não existe, ou utilizador sem permissões para operar sobre a casa

Exemplo:

Comando: **RH Casa1 M1**

Resposta: OK, 70 (long), seguido de *70 bytes de dados.*

Comando: **RH Casa4 M1**

Resposta: NOHM # esta casa não existe

Comando: **RH Casa1 M5**

Resposta: NOD # dispositivo não existe

Comando: **RH Casa2 M1**

Resposta: NOPERM # sem permissões

Servidor

O servidor mantém um **ficheiro de texto** com os **utilizadores** do sistema e as suas respetivas passwords. Este ficheiro deverá ser utilizado quando o cliente se tenta autenticar no servidor.

O servidor mantém um **ficheiro de texto** com as **casas** existentes, nomeadamente o seu nome (Casa1, Casa2, …) e dispositivos inteligentes que lhe estão associados. Esse ficheiro deverá também manter a lista de utilizadores que têm permissões de gestão (total ou parcial) de cada casa. Este ficheiro deverá ser utilizado quando o cliente tenta gerir a casa (i.e., ler dados do servidor ou enviar comandos para o servidor para gerir os dispositivos de uma casa). Também, o servidor mantém um **ficheiro de texto** com os últimos estados/temporizações de cada dispositivo, bem como os ficheiros de log de cada dispositivo.

O servidor deve atestar a aplicação cliente, ou seja, que a aplicação que estabeleceu conexão com ele é, de facto, a ***SpertaClient.*** Para tal, o servidor recebe o tamanho desta (vindo da própria aplicação cliente) e o valida com a informação que possui localmente sobre a aplicação. O servidor mantém um

ficheiro de texto com o nome da aplicação e o respetivo tamanho (ex., SpertaClient:2734). Se a atestação falhar, o servidor enviará uma mensagem de falha de atestação e o cliente imprimirá a mensagem "`ATTESTATION FAILED`". Se a atestação for bem-sucedida, o servidor enviará uma mensagem de sucesso e o cliente imprimirá a mensagem "`ATTESTATION OK`".

Após a verificação da atestação da SpertaClient, o utilizador autentica-se no servidor. Caso um utilizador se tente autenticar, e não esteja registado no servidor, o Servidor fará o registo do novo utilizador, adicionando o *user-id* e a respetiva *password* ao ficheiro de utilizadores do servidor. Devem também ser inicializadas as estruturas de dados que mantêm a informação relativa a cada utilizador.

Quando um utilizador cria uma casa, o servidor cria a estrutura da Fig 2 para a nova casa, onde cada seção contém um contador iniciado a 1. Assim, cada vez que o comando **`RD <hm> <s>`** é usado para registar um dispositivo numa dada seção `<s>` de uma casa `<hm>` (ex., **`RD Casa1 P`**), o contador associado àquela seção, daquela casa é incrementado, e consequentemente, o dispositivo é criado e registado no ficheiro da casa. Esta estrutura e contadores terá de ser mantida num **ficheiro de texto** para manter a informação de contadores.

4 Entrega

O prazo de entrega é no dia **27 de março, até às 23:59 horas**.

O código desta primeira fase do trabalho deve ser entregue de acordo com as seguintes regras:

- Para entregar o trabalho, é necessário criar um **ficheiro zip** com o nome **SegC-grupoXX-proj1-fase1.zip**, onde **XX** é o número do grupo, contendo:
  - ✓ o código do trabalho;
  - ✓ os ficheiros jar (cliente e servidor) para execução do projeto;
  - ✓ um readme (txt) indicando **como compilar** e **como executar** o projeto, indicando também as limitações do trabalho.
- O ficheiro **zip** é submetido através da atividade disponibilizada para esse efeito na página da disciplina no Moodle. Apenas um dos elementos do grupo deve submeter. Se existirem várias submissões do mesmo grupo, será considerada a mais recente.
- Não serão aceites trabalhos enviados por email. Se não se verificar algum destes requisitos, o trabalho é considerado não entregue.

5 Autoavaliação de contribuições

Cada aluno tem de preencher, no Moodle, um formulário de autoavaliação das contribuições individuais de cada elemento do grupo para o projeto. Por exemplo, se todos os elementos colaboraram de forma idêntica, bastará que todos indiquem que cada um contribuiu 33%. Aplicam-se as seguintes regras e penalizações:

- Alunos que não preencham o formulário **sofrem uma penalização de 10% na classificação.**
- Caso existam assimetrias significativas entre as respostas de cada elemento do grupo, o grupo poderá ser chamado para as explicar.
- Se as contribuições individuais forem diferentes, isso será refletido na nota de cada elemento do grupo, levando à atribuição de notas individuais diferentes.

O prazo de preenchimento é o mesmo que o da entrega do projeto (**27 de março, até às 23:59 horas**).